PharmaFinder

Digital prescriptions linked to live pharmacy stock in Uasin Gishu County, Kenya

Group 3, ALU Foundations Project, technical presentation

Pauline Miyienda
Frontend, project manager

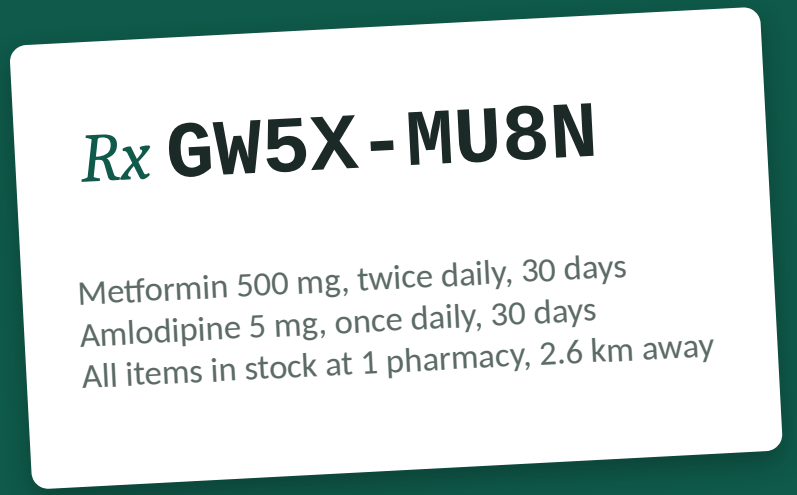
Marion Gitau
Backend

Emmanuel Masambu
Database architect

Sylvie Uwera
Frontend

Ahmed Osman
UI/UX design

Batsinda Benoit
Backend



Two problems, one workflow

21%

of handwritten prescriptions were legible at a Uasin Gishu public hospital, even after an improvement project (Amdany & Kiprop, 2023)

16.1%

availability of NCD medicines at Kenyan dispensaries across 8 counties, so patients travel between pharmacies (Ashigbie et al., 2020)

How PharmaFinder answers both



Doctor issues a structured prescription

Every field validated by the API; unique code, expiry date, alert to the patient



Patient finds pharmacies that can fill it

Ranked by full stock coverage, then distance and price, on a Leaflet map



Patient shares it with one pharmacy

Time-limited access the patient can revoke at any moment



Pharmacist verifies and dispenses

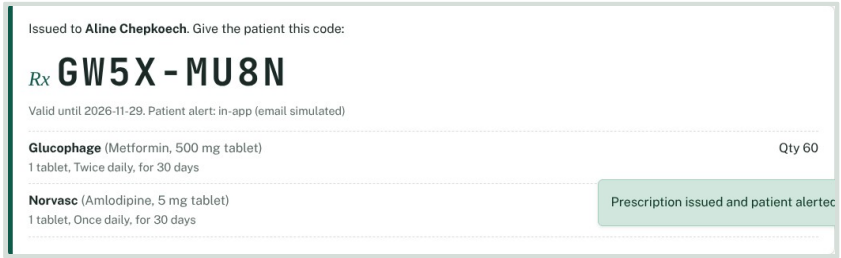
Stock falls in the same transaction; the code can never be used again

What is built, and what changed

Feature	Status	Evidence
Registration, login, four roles	**Working**	test_auth.py (9 tests)
Prescription form, unique code, issue alert	**Working**	test_prescriptions.py
Patient prescription history	**Working**	test_patient_sees_only_own…
Share with a pharmacy, revoke	**Working**	test_sharing.py (6 tests)
Medicine search, filters, sort, map	**Working**	test_stock_search.py
Stock management	**Working**	test_stock_search.py
Code verification and dispensing	**Working**	test_dispense.py (7 tests)
Admin dashboard and audit log	**Working**	test_admin_journey.py
Deployment on Vercel	**Ready, not deployed**	Steps in README

Changed from the proposal

- QR code replaced by an 8-character code
- Reservations and payment dropped
- Digital signature dropped
- No self-registration for doctors or pharmacists; the admin creates them



A prescription issued in the running app (synthetic data)

Technology choices

Python 3 + Flask

Small REST API with blueprints per role. Chosen over Django: we need an API, not an admin site or ORM.

MySQL 8

Linked data, foreign keys and transactions for dispensing. Chosen over MongoDB, which lacks relational joins and constraints.

HTML, CSS, JS + Bootstrap 5

No build step, so every team member can edit pages. Chosen over React for a 6-week build.

Leaflet.js + OpenStreetMap

Free and open, no API key or billing. Google Maps needs a paid key.

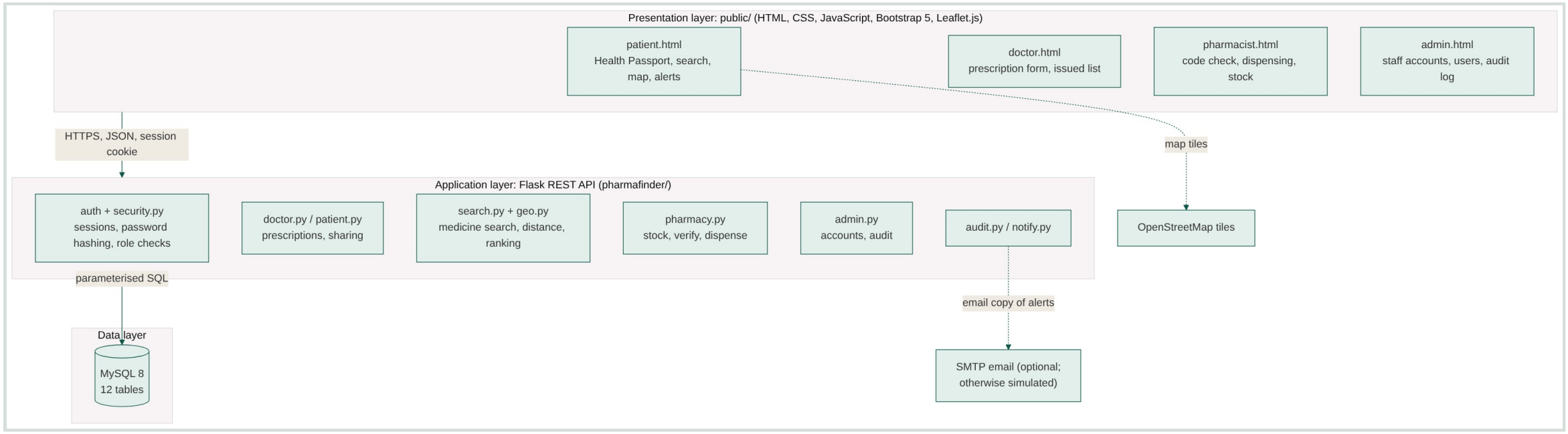
pytest + Postman

pytest for unit, integration and security tests on real MySQL; Postman for HTTP-level checks.

Git, GitHub, Vercel

Version control and pull requests; Vercel hosts Flask free as a serverless function.

Three-tier architecture



Presentation

4 role pages call the API with fetch() and escape all server data before display

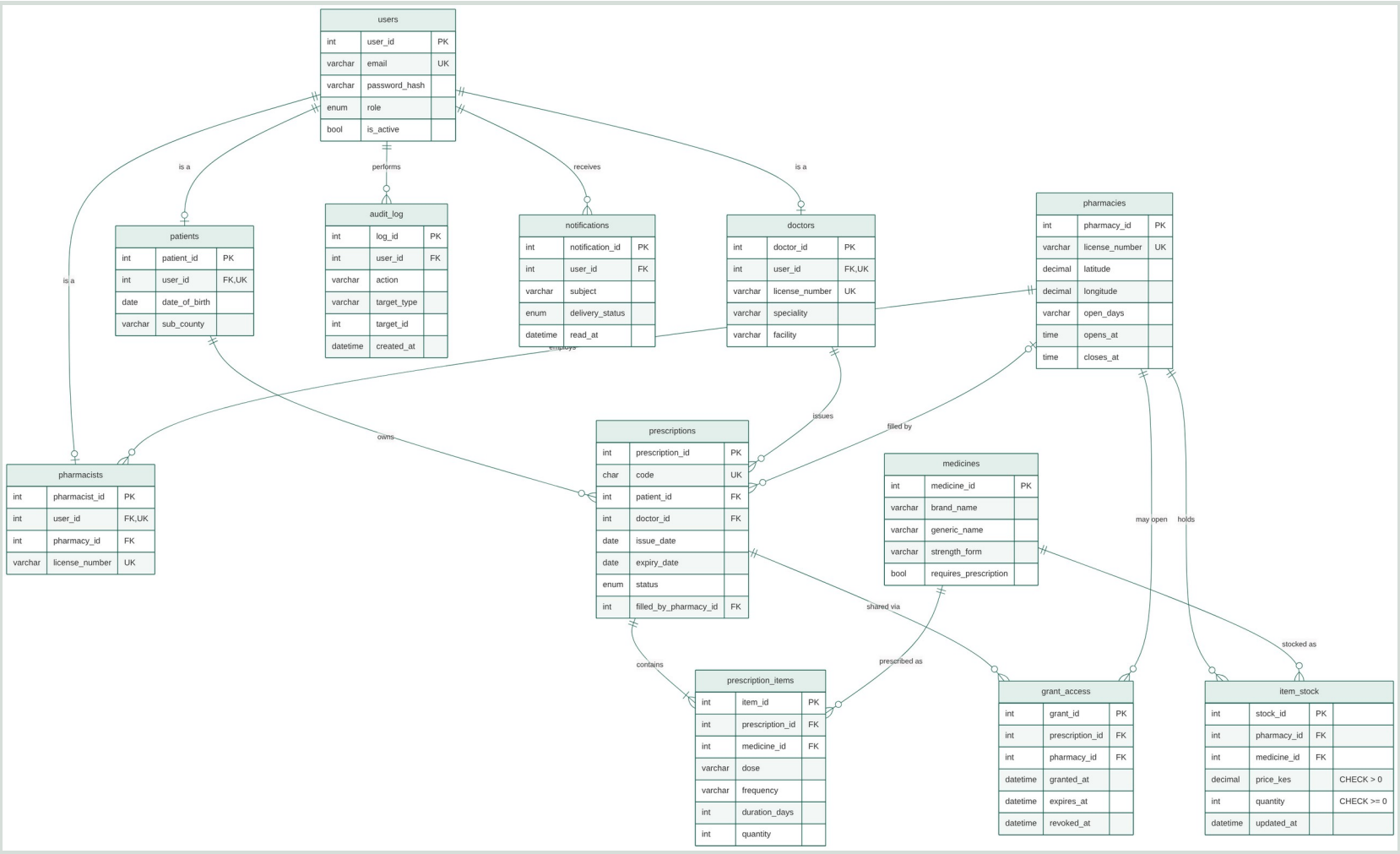
Application

Flask checks login, role and ownership on every request, validates input, writes the audit log

Data

MySQL is reached only by the API, with parameterised SQL and transactions

Database design: 12 tables



Keys and constraints

- Patient, Doctor, Pharmacist are 1:1 with User via a unique user_id
- Prescription code is UNIQUE
- CHECK quantity >= 0 and price_kes > 0 on stock
- UNIQUE (pharmacy, medicine) stock line
- Indexes on generic name and on grant lookups

Changed from proposal
Reservation replaced by notifications; added password hash, coordinates, generic name, revoked_at; 'update' column renamed (MySQL reserved word); prices in KES

Core logic: codes and single-use dispensing

Unguessable prescription codes (rx.py)

```
# No 0/O, 1/I/L, so codes are read
# correctly at the counter
CODE_ALPHABET = "ABCDEFGHJKMNPQRSTUVWXYZ23456789"

def generate_code():
    raw = "".join(secrets.choice(CODE_ALPHABET)
                  for _ in range(8))
    return f"{raw[:4]}-{raw[4:]}"
```

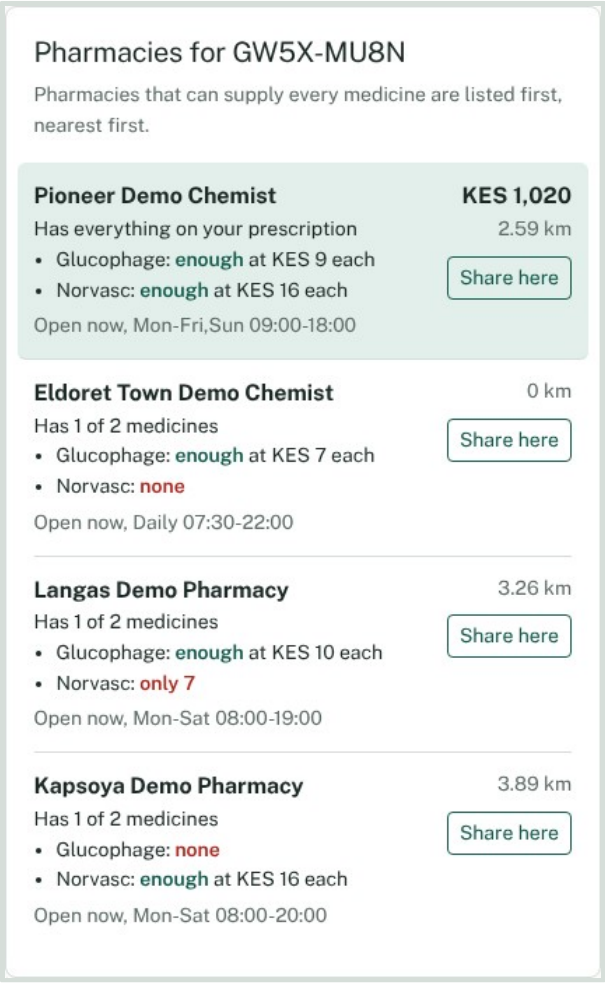
- 31^8 ≈ 850 billion possible codes, from a cryptographic random source
- UNIQUE in MySQL; on the rare clash the API retries
- 10 wrong codes in 15 minutes locks the pharmacist out (429)

Dispensing in one transaction (routes/pharmacy.py, abridged)

```
rx = query_one("SELECT ... FROM prescriptions"
               " WHERE code=%s FOR UPDATE", (code,))
# check grant, status, expiry, stock for all items
for i in items:
    cur.execute("UPDATE item_stock SET quantity ="
        " quantity - %s WHERE stock_id=%s"
        " AND quantity >= %s", (...))
cur.execute("UPDATE prescriptions SET status='filled'"
            " WHERE prescription_id=%s"
            " AND status='active'", (...))
if cur.rowcount != 1:
    raise ApiError("Already filled", 409)
log_action(...); notify(...); conn.commit()
```

Row locks and conditional updates: if anything fails, nothing changes

Search, ranking and the map



Real ranking from the app (synthetic data)

1. **Distance**

   Haversine great-circle distance from the patient (browser location, or Eldoret CBD in the demo) to each pharmacy

2. **Coverage**

   Each prescribed item is matched against the pharmacy's stock: enough only if quantity ≥ prescribed amount

3. **Rank**

   Can fill everything first, then more items covered, then nearest, then cheapest total

```
def key(p):                          # geo.py
    full = p["items_available"] == needed_count
    return (0 if full else 1, -p["items_available"],
            p["distance_km"], p["total_price_kes"])
```

Search by name: brand or generic, sort by distance, price or stock, filter by open now and distance. **Map:** Leaflet markers on OpenStreetMap tiles; stock times shown so patients see how fresh it is.

Security and data protection

Passwords

Hashed with scrypt and a random salt (Werkzeug); never stored in plain text

Roles and ownership

Checked on the server on every request: 401 not logged in, 403 wrong role, 404 not yours

Patient-controlled sharing

A pharmacy opens a prescription only during a 1–72 h grant; revoking blocks it instantly

Single use and expiry

Filled, expired or cancelled prescriptions cannot be dispensed; access closes after dispensing

Audit log

Logins, prescriptions, sharing, stock changes, dispensing and admin actions, with user and time

Attempt limits

5 failed logins or 10 wrong codes in 15 minutes returns 429

Not claimed: encryption of stored data, anonymised search and account deletion from the proposal are not implemented yet. HTTPS comes from Vercel; TLS to MySQL when configured.

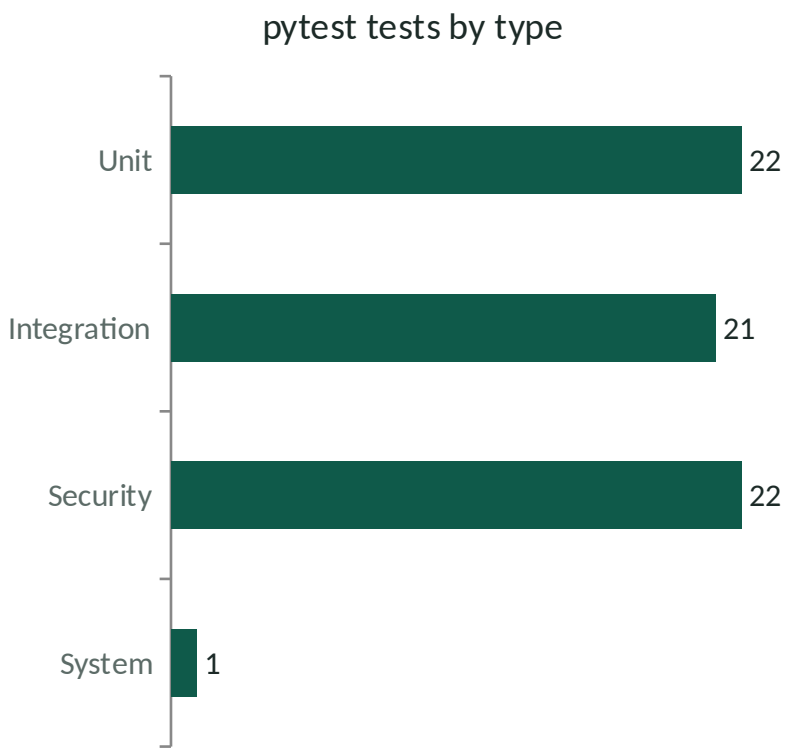
Code quality and testing

66 / 66

pytest tests passed, on real MySQL rebuilt before each test

30 / 30

Postman assertions passed across 22 API requests (Newman)



Example cases from the test report

Full journey: doctor issues, patient shares, pharmacist dispenses; stock 200 to 140 and 170

Code reuse: second dispense returns 409 and stock is unchanged

Invalid input: 10 bad prescriptions all rejected, none saved

Role escalation: signing up as doctor still creates a patient

Code quality: one Flask blueprint per role, shared helpers for validation, security and audit; PEP 8 style; every SQL query parameterised; generated test report in docs/TEST_REPORT.md.

Technical challenges and solutions

Challenge	Solution	Did it work?
Two pharmacists dispensing one code at once	SELECT … FOR UPDATE row locks and updates conditional on status	Concurrency test: one 200, one 409, stock reduced once
Vercel has no background jobs, memory or disk between requests	Expiry applied whenever prescriptions are read; attempt limits counted from the audit log	Works locally as stateless code; live Vercel deploy still to do
Codes without QR can be guessed or misread	Random 8-character code without look-alike characters; 10 failures in 15 min blocked	Rate-limit test passes; lower-case and spaced input accepted
Proposal schema could not support the features	Added password hash, coordinates, generic name, revoked_at; renamed the reserved-word column	All 12 tables created from schema.sql in every test run

Feedback integration

We have not yet received formal reviewer feedback on the implementation. Changes so far came from the team's own design review of the proposal.

Design review of the proposal

Finding: schema missing password hash, pharmacist-pharmacy link, coordinates

Change: columns added (slide 6)

Finding: anyone could register as a doctor

Change: only patients self-register; admin creates staff

Finding: QR expiry and reservation timer were mixed up

Change: scope simplified to one code with patient-controlled access

How we will handle feedback

- Log each comment as a card on our board
- Fix it on a branch with a pull request
- Add or update a test that proves the fix
- Re-run pytest and Postman before merging

Next technical steps

Owner	Task
Marion Gitau	Deploy the Flask API and pages to Vercel; set environment variables
Emmanuel Masambu	Create the Aiven MySQL database, run the seed, set up backups
Batsinda Benoit	Run the Postman collection against the live URL; connect real email for alerts
Sylvie Uwera	Test on phones over slow connections; capture screenshots with live map tiles
Ahmed Osman	Usability test with 5 users using the proposal's 5-point Likert scale
Pauline Miyienda	Coordinate review and merge of pull request #1; track tasks on the board

Milestones

Week 5
Live on Vercel, all tests re-run against it

Week 6
User evaluation and security testing, per the proposal timeline

After the course
Pilot with real Eldoret pharmacies

References and acknowledgements

Research

Amdany, H., & Kiprop, J. W. (2023). Handwritten prescription practices in a public hospital in Uasin Gishu County, Kenya. JBI Evidence Implementation, 21(4), 345–354.

Ashigbie, P. G., et al. (2020). Availability and prices of medicines for non-communicable diseases in Kenya. BMJ Open, 10(5), e035132.

Kommey, B., et al. (2024). MeDASH: checking the availability of pharmaceutical drugs in proximity using QR codes. SPEKTA, 5(2), 144–167.

Wondmagegn, M. T., et al. (2023). A framework for locating prescribed medication at pharmacies. Integrated Pharmacy Research and Practice, 12, 127–136.

Ariaga, A., et al. (2023). A scoping review of legibility of hand-written prescriptions and drug-orders. Expert Review of Clinical Pharmacology, 16(7), 617–621.

Toroitich, A. M., et al. (2022). Patients access to medicines: a critical review of the healthcare system in Kenya. Risk Management and Healthcare Policy, 15, 361–374.

Libraries and services

- Flask 3, Werkzeug, PyMySQL (Pallets; PyMySQL contributors)
- Bootstrap 5.3, Leaflet 1.9.4
- Map data © OpenStreetMap contributors
- pytest, Postman and Newman, Mermaid
- Vercel, Aiven for MySQL (deployment)

Thank you

To our ALU facilitators. Development was assisted by Claude (Anthropic), as permitted for this project. Code: github.com/EMANU02004/Pharma